DSA Lab Sheet – 07: Trees

Subject: Data Structure I (BTCS209P) | Language: C++ | Semester: III | Session: 2026–2027

Q8. Create a Binary Tree and Display Its Structure

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *left, *right;
    Node(int x) : data(x), left(NULL), right(NULL) {}
};

int main() {
    Node *root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);

    cout << root->data << endl;
    cout << root->left->data << " " << root->right->data;
}
```

Output: 1 \ 2 3 | Time: O(1) for fixed tree | Space: O(n)

Q9. Inorder Traversal

Order: Left → Root → Right

```
#include <iostream>
using namespace std;

struct Node {
    int d; Node *l, *r;
    Node(int x) : d(x), l(NULL), r(NULL) {}
};

void inorder(Node* n) {
    if (n) {
        inorder(n->l);
        cout << n->d << " ";
        inorder(n->r);
    }
}

int main() {
    Node* r = new Node(2);
    r->l = new Node(1);
    r->r = new Node(3);
    inorder(r);
}
```

Output: 1 2 3 | Time: O(n) | Space: O(h)

Q10. Preorder Traversal

Order: Root → Left → Right

```
#include <iostream>
using namespace std;

struct Node {
    int d; Node *l, *r;
    Node(int x) : d(x), l(NULL), r(NULL) {}
};

void preorder(Node* n) {
    if (n) {
        cout << n->d << " ";
        preorder(n->l);
        preorder(n->r);
    }
}

int main() {
    Node* r = new Node(2);
    r->l = new Node(1);
    r->r = new Node(3);
```

```
    preorder(r);
}
```

Output: 2 1 3 | Time: O(n) | Space: O(h)

Q11. Postorder Traversal

Order: Left → Right → Root

```
#include <iostream>
using namespace std;

struct Node {
    int d; Node *l, *r;
    Node(int x) : d(x), l(NULL), r(NULL) {}
};

void postorder(Node* n) {
    if (n) {
        postorder(n->l);
        postorder(n->r);
        cout << n->d << " ";
    }
}

int main() {
    Node* r = new Node(2);
    r->l = new Node(1);
    r->r = new Node(3);
    postorder(r);
}
```

Output: 1 3 2 | Time: O(n) | Space: O(h)

Q12. All Three Tree Traversals

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l, *r;
    N(int x) : d(x), l(NULL), r(NULL) {}
};

void in(N* n) { if(n) { in(n->l); cout<<n->d<<" "; in(n->r); } }
void pre(N* n) { if(n) { cout<<n->d<<" "; pre(n->l); pre(n->r); } }
void post(N* n) { if(n) { post(n->l); post(n->r); cout<<n->d<<" "; } }

int main() {
    N* r = new N(2);
    r->l = new N(1); r->r = new N(3);
    cout << "Inorder: "; in(r);
    cout << "
Preorder: "; pre(r);
    cout << "
Postorder: "; post(r);
}
```

Output: Inorder: 1 2 3 \ Preorder: 2 1 3 \ Postorder: 1 3 2 | Time: O(n) per traversal | Space: O(h)

Q13. Create a BST by Inserting n Elements

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

N* insert(N* n,int x) {
    if(!n) return new N(x);
    if(x<n->d) n->l=insert(n->l,x);
    else if(x>n->d) n->r=insert(n->r,x);
    return n;
}

void inorder(N* n) {
```

```
    if(n) { inorder(n->l); cout<<n->d<<" "; inorder(n->r); }
}

int main() {
    N* r=NULL;
    int a[]={50,30,70,20,40};
    for(int x:a) r=insert(r,x);
    inorder(r);
}
```

Output: 20 30 40 50 70 | Time: O(n log n) avg, O(n²) worst | Space: O(h)

Q14. Search for a Key in BST

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

N* ins(N* n,int x) {
    if(!n) return new N(x);
    if(x<n->d) n->l=ins(n->l,x);
    else if(x>n->d) n->r=ins(n->r,x);
    return n;
}

bool search(N* n,int x) {
    if(!n) return false;
    if(n->d==x) return true;
    return x<n->d ? search(n->l,x) : search(n->r,x);
}

int main() {
    N* r=NULL;
    for(int x:{50,30,70}) r=ins(r,x);
    cout << (search(r,30)?"Found":"Not Found");
}
```

Output: Found | Time: O(log n) avg, O(n) worst | Space: O(h)

Q15. Insert and Display BST Inorder

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

N* ins(N* n,int x) {
    if(!n) return new N(x);
    if(x<n->d) n->l=ins(n->l,x);
    else if(x>n->d) n->r=ins(n->r,x);
    return n;
}

void in(N* n) {
    if(n) { in(n->l); cout<<n->d<<" "; in(n->r); }
}

int main() {
    N* r=NULL;
    for(int x:{40,20,60,10,30}) r=ins(r,x);
    in(r);
}
```

Output: 10 20 30 40 60 | Time: O(n log n) | Space: O(h)

Q16. Delete a Leaf Node from BST

```
#include <iostream>
using namespace std;

struct N {
```

```
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

N* ins(N* n,int x) {
    if(!n) return new N(x);
    if(x<n->d) n->l=ins(n->l,x);
    else if(x>n->d) n->r=ins(n->r,x);
    return n;
}

N* del(N* n,int x) {
    if(!n) return n;
    if(x<n->d) n->l=del(n->l,x);
    else if(x>n->d) n->r=del(n->r,x);
    else if(!n->l && !n->r) { delete n; return NULL; }
    return n;
}

void in(N* n) { if(n) { in(n->l); cout<<n->d<<" "; in(n->r); } }

int main() {
    N* r=NULL;
    for(int x:{50,30,70}) r=ins(r,x);
    r=del(r,30); in(r);
}
```

Output: 50 70 | Time: O(log n) avg | Space: O(h)

Q17. Delete a Node Having One Child

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

N* ins(N* n,int x) {
    if(!n) return new N(x);
    if(x<n->d) n->l=ins(n->l,x);
    else if(x>n->d) n->r=ins(n->r,x);
    return n;
}

N* del(N* n,int x) {
    if(!n) return n;
    if(x<n->d) n->l=del(n->l,x);
    else if(x>n->d) n->r=del(n->r,x);
    else if(!n->l || !n->r) {
        N* t = n->l ? n->l : n->r;
        delete n; return t;
    }
    return n;
}

void in(N* n) { if(n) { in(n->l); cout<<n->d<<" "; in(n->r); } }

int main() {
    N* r=NULL;
    for(int x:{50,30,20}) r=ins(r,x);
    r=del(r,30); in(r);
}
```

Output: 20 50 | Time: O(log n) avg | Space: O(h)

Q18. Delete a Node Having Two Children

Uses inorder successor (smallest in right subtree).

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};
```

```
N* ins(N* n,int x) {
    if(!n) return new N(x);
    if(x<n->d) n->l=ins(n->l,x);
    else if(x>n->d) n->r=ins(n->r,x);
    return n;
}

N* del(N* n,int x) {
    if(!n) return n;
    if(x<n->d) n->l=del(n->l,x);
    else if(x>n->d) n->r=del(n->r,x);
    else {
        if(!n->l || !n->r) {
            N* t = n->l ? n->l : n->r;
            delete n; return t;
        }
        N* t = n->r;
        while(t->l) t = t->l;
        n->d = t->d;
        n->r = del(n->r, t->d);
    }
    return n;
}

void in(N* n) { if(n) { in(n->l); cout<<n->d<<" "; in(n->r); } }

int main() {
    N* r=NULL;
    for(int x:{50,30,70,60,80}) r=ins(r,x);
    r=del(r,50); in(r);
}
```

Output: 30 60 70 80 | Time: O(log n) avg | Space: O(h)

Q19. Calculate Height of a Binary Tree

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

int height(N* n) {
    if(!n) return 0;
    int a = height(n->l), b = height(n->r);
    return 1 + (a > b ? a : b);
}

int main() {
    N* r = new N(1);
    r->l = new N(2); r->r = new N(3);
    cout << "Height = " << height(r);
}
```

Output: Height = 2 | Time: O(n) | Space: O(h)

Q20. Count Total Nodes

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

int countNodes(N* n) {
    if(!n) return 0;
    return 1 + countNodes(n->l) + countNodes(n->r);
}

int main() {
    N* r = new N(1);
    r->l = new N(2); r->r = new N(3);
    cout << "Nodes = " << countNodes(r);
}
```

Output: Nodes = 3 | Time: O(n) | Space: O(h)

Q21. Count Leaf Nodes

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

int leaves(N* n) {
    if(!n) return 0;
    if(!n->l && !n->r) return 1;
    return leaves(n->l) + leaves(n->r);
}

int main() {
    N* r = new N(1);
    r->l = new N(2); r->r = new N(3);
    cout << "Leaf nodes = " << leaves(r);
}
```

Output: Leaf nodes = 2 | Time: O(n) | Space: O(h)

Q22. Menu-Driven BST

```
#include <iostream>
using namespace std;

struct N {
    int d; N *l,*r;
    N(int x):d(x),l(NULL),r(NULL){}
};

N* ins(N* n,int x) {
    if(!n) return new N(x);
    if(x<n->d) n->l=ins(n->l,x);
    else if(x>n->d) n->r=ins(n->r,x);
    return n;
}

bool search(N* n,int x) {
    if(!n) return false;
    if(n->d==x) return true;
    return x<n->d?search(n->l,x):search(n->r,x);
}

N* del(N* n,int x) {
    if(!n) return n;
    if(x<n->d) n->l=del(n->l,x);
    else if(x>n->d) n->r=del(n->r,x);
    else {
        if(!n->l||!n->r) {
            N* t=n->l?n->l:n->r;
            delete n; return t;
        }
        N* t=n->r;
        while(t->l) t=t->l;
        n->d=t->d; n->r=del(n->r,t->d);
    }
    return n;
}

void in(N* n) { if(n) { in(n->l); cout<<n->d<<" "; in(n->r); } }
void pre(N* n) { if(n) { cout<<n->d<<" "; pre(n->l); pre(n->r); } }
void post(N* n) { if(n) { post(n->l); post(n->r); cout<<n->d<<" "; } }
int height(N* n) { if(!n) return 0; int a=height(n->l),b=height(n->r); return 1+(a>b?a:b); }
int countNodes(N* n) { return n?1+countNodes(n->l)+countNodes(n->r):0; }

int main() {
    N* r=NULL; int c,x;
    do {
        cout << "
1.Insert 2.Search 3.Inorder 4.Preorder
5.Postorder 6.Delete 7.Height 8.Count 9.Exit
Choice: ";
        cin >> c;
```

```
        switch(c) {
            case 1: cin>>x; r=ins(r,x); break;
            case 2: cin>>x; cout<<(search(r,x)?"Found":"Not Found"); break;
            case 3: in(r); break;
            case 4: pre(r); break;
            case 5: post(r); break;
            case 6: cin>>x; r=del(r,x); break;
            case 7: cout << height(r); break;
            case 8: cout << countNodes(r); break;
        }
    } while(c!=9);
}
```

Time: Insert/Search/Delete O(log n) avg, Traversals/Height/Count O(n) | Space: O(n)